# Surface Module Intelligence System

**Author:** Andrews Ferreira
**Module:** `modules.surface.intelligence`

## Overview

The Surface module intelligence system transforms basic port scanning and service detection into an intelligent attack surface analyzer. It provides:

- **Service Intelligence Database** – Comprehensive knowledge base of services, attack context, and tools
- **Service Normalization** – Consistent naming across all tools and outputs
- **Correlation Engine** – Links ports ↔ services ↔ URLs into a unified map
- **Deduplication** – Merges duplicate findings from multiple detection methods
- **Confidence Scoring** – Multi-signal scoring based on port, banner, version, and detection count
- **Intelligent Prioritization** – Category-grouped, actionable recommendations

---

## Architecture

```
modules/surface/intelligence/
├── __init__.py                 # Package exports
├── service_intelligence.py     # Service intelligence database
├── service_normalizer.py       # Name normalization engine
├── correlation_engine.py       # Port ↔ Service ↔ URL correlation
├── deduplicator.py            # Duplicate detection & merging
├── confidence_scorer.py        # Multi-signal confidence scoring
└── attack_prioritizer.py       # Intelligent prioritization engine
```

### Data Flow

```
Port Discovery (Phase 1)
    └ raw ports
Service Fingerprint (Phase 2)
    └ raw services + HTTP probes
          ||
          ▼
Vector Correlation (Phase 3) ← Intelligence Engine
    ├ Deduplication → Remove duplicate entries
    ├ Normalization → Canonical service names
    ├ Correlation   → Unified AttackSurfaceMap
    └ Confidence    → Multi-signal scoring
          ||
          ▼
Prioritization (Phase 4) ← Attack Prioritizer
    ├ Category grouping (AD, Database, Web, Remote Access...)
    ├ Quick wins identification
    ├ High-value target highlighting
    └ Action plan generation
```

# Components

## 1. Service Intelligence Database ( `service_intelligence.py` )

Comprehensive port-to-service mapping with offensive context.

**ServiceProfile fields:**

| Field | Description |
|---|---|
| `canonical_name` | Standardized name (e.g., `smb` , `rdp` , `http` ) |
| `display_name` | Human-readable name (e.g., `SMB` , `RDP` ) |
| `description` | Service description |
| `category` | Attack category: `ad` , `web` , `database` , `remote_access` , `file_sharing` , `mail` , `monitoring` , `misc` |
| `default_ports` | Known default ports |
| `aliases` | All nmap/banner name variations |
| `attack_context` | What attacks are possible |
| `common_tools` | Pentesting tools for this service |
| `next_steps` | Suggested investigation steps |
| `high_value` | Is this a high-value target? |
| `cleartext` | Does it use cleartext? |
| `default_creds_common` | Are default creds common? |

**Example entry:**

```
ServiceProfile(
    canonical_name="smb",
    display_name="SMB",
    category="ad",
    default_ports=(445, 139),
    aliases=("microsoft-ds", "cifs", "netbios-ssn", "samba"),
    attack_context="File sharing, credential harvesting, lateral movement, relay attacks",
    common_tools=("enum4linux-ng", "smbclient", "crackmapexec", "smbmap"),
    next_steps=(
        "Test for null session access",
        "Enumerate shares and permissions",
        "Check SMB signing status",
        "Test for known CVEs (EternalBlue, PrintNightmare)",
    ),
    high_value=True,
    default_creds_common=True,
)
```

**Covered services:** SSH, RDP, VNC, Telnet, HTTP, HTTPS, SMB, LDAP, LDAPS, Kerberos, DNS, MS-RPC, WinRM, MSSQL, MySQL, PostgreSQL, Oracle, Redis, MongoDB, Elasticsearch, FTP, TFTP, NFS, SMTP, POP3, IMAP, SNMP, IPMI, Docker, Kubernetes

## 2. Service Normalizer ( `service_normalizer.py` )

Resolves naming inconsistencies:

| Original | Normalized |
|---|---|
| `microsoft-ds` | `smb` |
| `ms-wbt-server` | `rdp` |
| `http-proxy` | `http` |
| `ssl/http` | `https` |
| `kerberos-sec` | `kerberos` |
| `epmap` | `msrpc` |
| `ms-sql-s` | `mssql` |
| `domain` | `dns` |

**Resolution priority:**
1. Direct alias match in intelligence DB
2. SSL/TLS prefix stripping → alias match
3. Port-based resolution (fallback)
4. Return cleaned original if no match

## 3. Correlation Engine ( `correlation_engine.py` )

Builds a unified `AttackSurfaceMap`:

- **Port scan data** → normalized service with `port_scan` detection
- **Version scan data** → merged with version info, `version_scan` detection
- **HTTP probe data** → linked to `http` / `https` service with URLs and technologies
- Each service enriched with intelligence profile data

**Output:** `CorrelatedService`


```
{
    "canonical_name": "smb",
    "display_name": "SMB",
    "ports": [139, 445],
    "best_version": "3.1.1",
    "urls": [],
    "detection_methods": ["port_scan", "version_scan"],
    "category": "ad",
    "attack_context": "File sharing, credential harvesting...",
    "next_steps": ["Test for null session access", ...],
    "common_tools": ["enum4linux-ng", "smbclient", ...],
    "high_value": true,
    "confidence": 0.70
}
```


## 4. Deduplicator ( `deduplicator.py` )

Merges entries from different detection methods:

**Before deduplication:**

```
Port 445/tcp: microsoft-ds (from port scan, no version)
Port 445/tcp: microsoft-ds Samba 4.15.2 (from version scan)
```

**After deduplication:**

```
Port 445/tcp: smb (Samba 4.15.2) [detected by: port_scan, version_scan]
```

**Merge strategy:**
- Prefer longer/more specific string values
- Merge lists (union)
- Accumulate detection methods
- Track all sources

## 5. Confidence Scorer ( `confidence_scorer.py` )

Multi-signal scoring:

| Signal | Weight | Description |
|---|---|---|
| `port_match` | 0.25 | Service on its known default port |
| `banner_match` | 0.25 | Banner/product confirms service identity |
| `version_detected` | 0.20 | Specific version was identified |
| `multi_detection` | 0.20 | 2+ detection methods confirmed |
| `http_confirmed` | 0.10 | HTTP probe confirmed web service |

**Confidence labels:**
| Score | Label |
|---|---|
| ≥ 0.80 | `confirmed` |
| ≥ 0.60 | `high` |
| ≥ 0.40 | `medium` |
| < 0.40 | `low` |

**Example:**

```
SSH on port 22: confirmed (90%)
  Signals: default port detected, banner confirmed, version identified, 2 detection
methods
```

### 6. Attack Prioritizer ( `attack_prioritizer.py` )

Produces actionable intelligence instead of just numeric scores:

**Priority calculation:**

| Factor | Weight |
|---|---|
| High-value target | +3.0 |
| AD category | +2.5 |
| Default creds common | +2.0 |
| Database category | +2.0 |
| Cleartext protocol | +1.5 |
| Remote access category | +1.5 |
| Version known | +1.0 |
| Web category | +1.0 |

**Priority levels:**

| Score | Level |
|---|---|
| ≥ 6.0 | `critical` |
| ≥ 4.0 | `high` |
| ≥ 2.0 | `medium` |
| < 2.0 | `low` |

**Output includes:**
- **Ranked targets** with rationale, next steps, and tools
- **Category groups** (AD, Database, Remote Access, Web, etc.)
- **Quick wins** (default creds, cleartext, no-auth services)
- **High-value targets** (DCs, databases, admin interfaces)
- **Executive summary** with actionable overview
- **Action plan** (Markdown file)

---

## BEFORE / AFTER Examples

### Correlation

**BEFORE (old vector_correlation.py):**

```
Port 445/tcp: microsoft-ds → "Investigate: SMB signing, null sessions"
Port 139/tcp: netbios-ssn  → (not in HIGH_VALUE_SERVICES, ignored)
Port 80/tcp: http          → (not linked to httpx URL discoveries)
```

**AFTER (new intelligence engine):**

```
SMB [HIGH VALUE] → Ports: 139, 445
  Category: Active Directory
  Context: File sharing, credential harvesting, lateral movement, relay attacks
  Detection: port_scan, version_scan (confirmed, 90%)
  Next: 1) Test null sessions  2) Enumerate shares  3) Check signing  4) Test CVEs
  Tools: enum4linux-ng, smbclient, crackmapexec

HTTP [HIGH VALUE] → Ports: 80
  Category: Web
  URLs: http://10.10.10.1/ (title: "Admin Panel")
  Technologies: Apache 2.4.51, PHP 8.1
  Detection: port_scan, version_scan, http_probe (confirmed, 95%)
  Next: 1) Directory brute-force  2) Nuclei scan  3) Check admin panels
```

### Service Normalization

**BEFORE:** `microsoft-ds`, `ms-wbt-server`, `http-proxy`, `ssl/http`
**AFTER:** `smb`, `rdp`, `http`, `https`

### Deduplication

**BEFORE:** 3 entries for port 445 (port scan + version scan + service detection)
**AFTER:** 1 entry with merged data from all 3 sources

### Prioritization

**BEFORE:**

```
Priority #1: microsoft-ds on port 445 (score: 3.0)
Priority #2: ssh on port 22 (score: 1.5)
```

**AFTER:**

```
## Quick Wins (Start Here)
- Redis (port 6379) — test unauthenticated access
- FTP (port 21) — test anonymous access

## Active Directory
#1 SMB [CRITICAL] — Ports: 139, 445 — Samba 4.15.2
   Rationale: high-value target; commonly has default credentials; AD service
   Next: Test null sessions → Enumerate shares → Check signing
   Tools: enum4linux-ng, smbclient, crackmapexec

## Databases
#3 MSSQL [HIGH] — Port: 1433
   Rationale: high-value target; commonly has default credentials; database service
   Next: Test sa:(blank) → Check xp_cmdshell → Enumerate databases
```

---

## Extending the Intelligence Database

To add a new service:

```
# In service_intelligence.py → _build_database()
ServiceProfile(
    canonical_name="newservice",
    display_name="New Service",
    description="Description of the service",
    category="misc",  # or ad, web, database, remote_access, etc.
    default_ports=(12345,),
    aliases=("alt-name", "other-name"),
    attack_context="What attacks are possible",
    common_tools=("tool1", "tool2"),
    next_steps=(
        "First investigation step",
        "Second investigation step",
    ),
    high_value=False,
    cleartext=False,
    default_creds_common=False,
)
```

The service will automatically be:
- Normalized from any alias
- Resolved by port number
- Enriched in correlation
- Scored for confidence
- Prioritized appropriately

---

# File Changes Summary

## New Files

- `modules/surface/intelligence/__init__.py`
- `modules/surface/intelligence/service_intelligence.py`
- `modules/surface/intelligence/service_normalizer.py`
- `modules/surface/intelligence/correlation_engine.py`
- `modules/surface/intelligence/deduplicator.py`
- `modules/surface/intelligence/confidence_scorer.py`
- `modules/surface/intelligence/attack_prioritizer.py`
- `modules/surface/SURFACE_INTELLIGENCE.md`

## Modified Files

- `modules/surface/phases/vector_correlation.py` – Complete rewrite using intelligence engine
- `modules/surface/phases/prioritization.py` – Complete rewrite using attack prioritizer
- `modules/surface/surface_module.py` – Updated data flow between phases 3→4, enhanced reports